

SiLapor

Sistem Pengaduan & Tracking Fasilitas Kampus

Dokumen Rancangan Konsep & Arsitektur Sistem

Tugas Besar Pemrograman III (Webservice)

Golang Fiber · PostgreSQL (Supabase) · Next.js

Daftar Isi

Daftar Isi	2
1. Latar Belakang Masalah	4
2. Tujuan Sistem	4
3. Aktor dan Hak Akses.....	4
4. Alur Sistem (Business Flow)	5
4.1 Alur Pelaporan oleh Mahasiswa	5
4.2 Alur Penanganan oleh Petugas	5
4.3 Alur Eskalasi Otomatis (Inti Nilai Tambah Sistem)	5
4.4 Alur Admin.....	5
5. Diagram Alur Status Laporan	6
6. Rancangan Struktur Database	6
6.1 Tabel: users.....	6
6.2 Tabel: kategori_fasilitas	6
6.3 Tabel: laporan	6
6.4 Tabel: riwayat_status.....	7
6.5 Diagram Relasi (ERD Sederhana).....	7
7. Rancangan Endpoint API	7
7.1 Autentikasi.....	7
7.2 Kategori Fasilitas (khusus admin).....	7
7.3 Laporan	8
8. Rancangan Tampilan Aplikasi (Frontend)	8
8.0 Stack Teknologi Frontend.....	8
8.1 Halaman Login & Register.....	9
8.2 Dashboard Mahasiswa	9
8.3 Form Buat Laporan.....	9
8.4 Halaman Detail Laporan & Timeline	9
8.5 Dashboard Admin / Petugas	10
8.6 Halaman Statistik (Admin)	10
8.7 Halaman Kelola Kategori (Admin).....	10
9. Fitur Nilai Tambah (Bonus)	10
10. Struktur Folder Project	10
10.1 Backend (Golang Fiber).....	10
10.2 Frontend (Next.js App Router).....	11
11. Kesimpulan	11

1. Latar Belakang Masalah

Di lingkungan kampus, kerusakan fasilitas seperti AC kelas yang mati, toilet bocor, proyektor rusak, atau kursi patah sering terjadi. Selama ini, mekanisme pelaporan masih mengandalkan cara informal: mahasiswa lapor lewat WhatsApp pribadi ke staf, mengisi Google Form yang jarang ditindaklanjuti, atau sekadar memberi tahu petugas kebersihan/keamanan secara lisan.

Masalah utama dari pendekatan ini adalah kurangnya transparansi dan akuntabilitas:

- Laporan sering “hilang” — tidak ada catatan resmi siapa yang melapor, kapan, dan ke mana laporan itu pergi.
- Mahasiswa tidak tahu apakah laporannya sedang diproses, diabaikan, atau sudah selesai.
- Tidak ada pembagian tanggung jawab yang jelas — siapa yang harus menangani kerusakan elektrik berbeda dengan siapa yang menangani sanitasi.
- Tidak ada mekanisme yang memastikan laporan lama tidak terus didiamkan.

SiLapor dirancang untuk menjawab masalah ini dengan menyediakan sistem pelaporan terstruktur yang otomatis mengarahkan laporan ke penanggung jawab yang tepat, memberi visibilitas status secara real-time kepada pelapor, dan secara otomatis meningkatkan prioritas laporan yang melewati batas waktu wajar penanganan (SLA).

2. Tujuan Sistem

1. Memberikan kanal pelaporan kerusakan fasilitas yang terstruktur dan terdokumentasi.
2. Mengotomatiskan distribusi laporan ke petugas sesuai kategori kerusakan.
3. Memberikan transparansi status laporan secara real-time kepada pelapor.
4. Mencegah laporan terbengkalai melalui mekanisme eskalasi otomatis berbasis SLA (Service Level Agreement).
5. Menyediakan data dan statistik bagi admin untuk evaluasi kinerja penanganan fasilitas kampus.

3. Aktor dan Hak Akses

Role	Deskripsi	Hak Akses Utama
Mahasiswa	Pengguna yang membuat laporan kerusakan	Membuat laporan baru; melihat daftar & status laporan miliknya sendiri; melihat riwayat/timeline status
Petugas	Penanggung jawab kategori fasilitas tertentu (mis. elektrik, sanitasi)	Melihat laporan sesuai kategori yang menjadi tanggung jawabnya; mengubah status laporan (ditugaskan → dikerjakan → selesai)
Admin	Pengelola sistem secara keseluruhan	Mengelola kategori fasilitas & penugasan petugas; melihat seluruh laporan; melihat dashboard statistik; mengelola akun pengguna

4. Alur Sistem (Business Flow)

4.1 Alur Pelaporan oleh Mahasiswa

6. Mahasiswa login ke sistem menggunakan akun terdaftar (NIM & password).
7. Mahasiswa membuka form “Buat Laporan Baru”, lalu mengisi: kategori fasilitas (dipilih dari dropdown, mis. Elektrik/Sanitasi/Furnitur/IT), lokasi (mis. “Ruang B201”), deskripsi kerusakan, dan foto bukti (opsional).
8. Sistem menyimpan laporan dengan status awal “dilaporkan” dan mencatat timestamp otomatis.
9. Sistem otomatis mengetahui petugas penanggung jawab berdasarkan kategori fasilitas yang dipilih (relasi kategori → petugas sudah ditentukan oleh admin sebelumnya), lalu status berubah menjadi “ditugaskan”.
10. Mahasiswa dapat memantau status laporan kapan saja melalui halaman “Laporan Saya”, lengkap dengan timeline riwayat perubahan status.

4.2 Alur Penanganan oleh Petugas

11. Petugas login dan melihat daftar laporan yang masuk sesuai kategori tanggung jawabnya.
12. Petugas memilih laporan, mengubah status menjadi “dikerjakan” saat mulai menangani.
13. Setelah selesai, petugas mengubah status menjadi “selesai” dan sistem mencatat tanggal_selesai secara otomatis.
14. Setiap perubahan status otomatis tersimpan ke tabel riwayat_status, sehingga seluruh proses penanganan dapat ditelusuri.

4.3 Alur Eskalasi Otomatis (Inti Nilai Tambah Sistem)

Setiap kategori fasilitas memiliki nilai SLA (mis. 48 jam) yang menandakan batas waktu wajar suatu laporan harus mulai ditangani. Mekanisme eskalasi berjalan melalui proses background (cron job sederhana memakai goroutine dan ticker di Golang) yang berjalan berkala, dengan logika berikut:

- Sistem memeriksa seluruh laporan berstatus “dilaporkan” atau “ditugaskan”.
- Jika selisih waktu sejak tanggal_lapor melebihi sla_jam dari kategorinya, sistem otomatis mengubah prioritas laporan menjadi “tinggi”.
- Sistem mencatat kejadian ini ke riwayat_status dengan keterangan “Eskalasi otomatis: melewati batas SLA”.
- Laporan berprioritas tinggi ditampilkan menonjol (badge merah) pada dashboard admin dan petugas, sehingga tidak ada laporan yang luput dari perhatian.

4.4 Alur Admin

15. Admin mengelola data kategori fasilitas dan menentukan petugas penanggung jawab tiap kategori.
16. Admin memantau seluruh laporan yang masuk dari semua kategori melalui dashboard terpusat.

17. Admin melihat statistik kinerja: jumlah laporan per kategori, rata-rata waktu penyelesaian, dan jumlah laporan yang sempat dieskalasi.

5. Diagram Alur Status Laporan

Status laporan bergerak melalui alur berikut:

DILAPORKAN → DITUGASKAN → DIKERJAKAN → SELESAI

Jika pada status DILAPORKAN atau DITUGASKAN laporan melewati batas SLA, prioritas otomatis berubah menjadi “tinggi” tanpa mengubah alur status utama — prioritas dan status berjalan sebagai dua atribut independen.

6. Rancangan Struktur Database

Struktur basis data menggunakan PostgreSQL (Supabase) dengan 4 tabel utama yang saling berelasi melalui foreign key, dikelola lewat GORM.

6.1 Tabel: users

Kolom	Tipe	Keterangan
id	uint	Primary Key, auto increment
nama	varchar	Nama lengkap pengguna
username / nim	varchar	Unique, dipakai untuk login
password	varchar	Hash bcrypt
role	varchar	admin / petugas / mahasiswa
created_at	timestamp	Otomatis

6.2 Tabel: kategori_fasilitas

Kolom	Tipe	Keterangan
id	uint	Primary Key
nama_kategori	varchar	Mis. Elektrik, Sanitasi, Furnitur, IT
petugas_id	uint	Foreign Key → users.id (penanggung jawab kategori)
sla_jam	int	Batas waktu wajar penanganan dalam jam, mis. 48

6.3 Tabel: laporan

Kolom	Tipe	Keterangan
id	uint	Primary Key
pelapor_id	uint	Foreign Key → users.id
kategori_id	uint	Foreign Key → kategori_fasilitas.id

lokasi	varchar	Mis. “Ruang B201”
deskripsi	text	Penjelasan kerusakan
foto_url	varchar	Opsional, link foto bukti
status	varchar	dilaporkan / ditugaskan / dikerjakan / selesai
prioritas	varchar	normal / tinggi (otomatis berubah via eskalasi)
tanggal_lapor	timestamp	Otomatis saat insert
tanggal_selesai	timestamp	Nullable, terisi saat status “selesai”

6.4 Tabel: riwayat_status

Kolom	Tipe	Keterangan
id	uint	Primary Key
laporan_id	uint	Foreign Key → laporan.id
status	varchar	Snapshot status pada waktu tersebut
keterangan	varchar	Mis. “Eskalasi otomatis karena melewati SLA”
waktu	timestamp	Otomatis

6.5 Diagram Relasi (ERD Sederhana)

users (1) — (N) kategori_fasilitas [sebagai penanggung jawab]

users (1) — (N) laporan [sebagai pelapor]

kategori_fasilitas (1) — (N) laporan

laporan (1) — (N) riwayat_status

Total terdapat 4 relasi foreign key, melampaui syarat minimal tugas besar (minimal 1 relasi), dan setiap relasi memiliki alasan bisnis yang jelas dan dapat dijelaskan saat presentasi.

7. Rancangan Endpoint API

7.1 Autentikasi

Method	Endpoint	Keterangan
POST	/register	Mendaftarkan user baru
POST	/login	Login, menghasilkan JWT
PUT	/changepassword	Mengubah password

7.2 Kategori Fasilitas (khusus admin)

Method	Endpoint	Keterangan
--------	----------	------------

GET	/api/kategori	Daftar semua kategori fasilitas
POST	/api/kategori	Tambah kategori baru
PUT	/api/kategori/:id	Ubah kategori / petugas penanggung jawab
DELETE	/api/kategori/:id	Hapus kategori

7.3 Laporan

Method	Endpoint	Keterangan
GET	/api/laporan	Daftar laporan (admin/petugas: semua sesuai akses; mahasiswa: miliknya saja)
GET	/api/laporan/:id	Detail satu laporan
POST	/api/laporan	Mahasiswa membuat laporan baru
PUT	/api/laporan/:id/status	Petugas mengubah status laporan
DELETE	/api/laporan/:id	Admin menghapus laporan
GET	/api/laporan/:id/riwayat	Melihat timeline riwayat status

Seluruh endpoint yang membutuhkan autentikasi dilindungi middleware JWT, dengan otorisasi tambahan berbasis role (admin/petugas/mahasiswa) untuk membedakan hak akses.

8. Rancangan Tampilan Aplikasi (Frontend)

8.0 Stack Teknologi Frontend

Frontend dibangun menggunakan Next.js (App Router) sebagai fondasi utama, dikombinasikan dengan beberapa pustaka pendukung agar tampilan terasa modern, variatif, dan interaktif — tidak sekadar dashboard CRUD biasa. Pemilihan Next.js sah digunakan karena ketentuan tugas secara eksplisit mengizinkan frontend bebas, termasuk Next.js.

Kebutuhan	Teknologi	Fungsi
Fondasi	Next.js (App Router) + Tailwind CSS	Routing, layout, dan styling utama
Komponen UI dasar	shadcn/ui	Table, dialog, dropdown, form, badge yang modern & mudah dikustom
Efek visual halaman login/landing	Aceternity UI / Magic UI	Animated background, hover effect, agar kesan pertama menarik
Dashboard statistik	Tremor + Recharts	Kartu statistik dan grafik interaktif untuk admin
Animasi transisi	Framer Motion	Transisi halaman, animasi badge status, modal slide-in
Notifikasi	Sonner	Toast notifikasi feedback aksi (berhasil/gagal)
Form & validasi	React Hook Form + Zod	Validasi form real-time tanpa reload

Tabel data laporan	TanStack Table	Sorting, filtering, pagination pada tabel laporan
--------------------	----------------	---

Catatan teknis penerapan Next.js agar tetap sesuai ketentuan tugas:

- Komponen yang mengakses localStorage atau Web API (token JWT, dsb) wajib diberi penanda “use client” di baris pertama file, karena App Router Next.js default berjalan sebagai Server Component.
- Fitur Next.js API Routes (folder app/api) tidak digunakan untuk logic utama — seluruh autentikasi dan CRUD tetap diproses oleh backend Golang Fiber, sesuai ketentuan tugas. Next.js di sini murni berperan sebagai dashboard/frontend.
- Environment variable untuk base URL API menggunakan prefix NEXT_PUBLIC_, mis. NEXT_PUBLIC_API_URL, agar dapat diakses di sisi client.
- Komunikasi ke backend tetap menggunakan Fetch API/Axios sesuai ketentuan tugas.
- Struktur folder mengikuti pola routing Next.js (folder = route) namun tetap menerapkan pemisahan Atomic Design pada folder components, sesuai anjuran tugas untuk proyek berbasis React.

8.1 Halaman Login & Register

Form input username/NIM dan password dengan latar animated gradient/spotlight effect (Aceternity UI) agar kesan pertama menarik. Validasi input ditangani React Hook Form + Zod dengan pesan error real-time. Setelah login berhasil, token JWT disimpan di localStorage dan pengguna diarahkan ke dashboard sesuai role-nya, disertai animasi transisi halaman dari Framer Motion.

8.2 Dashboard Mahasiswa

- Ringkasan jumlah laporan: total, sedang diproses, selesai.
- Tombol “+ Buat Laporan Baru” yang menonjol.
- Daftar laporan milik sendiri dalam bentuk kartu/list, masing-masing menampilkan status terkini dengan badge warna (kuning=diproses, hijau=selesai, merah=prioritas tinggi).

8.3 Form Buat Laporan

- Dropdown kategori fasilitas.
- Input lokasi (text).
- Textarea deskripsi kerusakan.
- Upload foto bukti (opsional, nilai tambah).
- Validasi frontend: kategori wajib dipilih, deskripsi minimal beberapa karakter, lokasi tidak boleh kosong.
- Feedback sukses/gagal menggunakan toast atau SweetAlert.

8.4 Halaman Detail Laporan & Timeline

Menampilkan informasi lengkap laporan beserta timeline vertikal riwayat status (data dari tabel riwayat_status), sehingga pelapor dapat melihat secara visual progres penanganan dari waktu ke waktu — termasuk catatan jika laporan sempat dieskalasi.

8.5 Dashboard Admin / Petugas

Tabel data utama dengan kolom: ID, Pelapor, Kategori, Lokasi, Status, Prioritas (6 kolom sesuai ketentuan tugas), dilengkapi:

- Fitur pencarian (cari berdasarkan lokasi/pelapor).
- Filter berdasarkan status dan prioritas.
- Tombol detail, ubah status, dan hapus (khusus admin).
- Badge merah menonjol untuk laporan berprioritas tinggi (hasil eskalasi otomatis).
- Loading state saat data sedang diambil dari API.

8.6 Halaman Statistik (Admin)

- Grafik jumlah laporan per kategori (bar chart).
- Grafik rata-rata waktu penyelesaian laporan.
- Jumlah laporan yang pernah dieskalasi dalam periode tertentu.
- Dibangun menggunakan Chart.js / Recharts sebagai nilai tambah (bonus).

8.7 Halaman Kelola Kategori (Admin)

Form CRUD sederhana untuk menambah/mengubah/menghapus kategori fasilitas beserta penentuan petugas penanggung jawab dan nilai SLA jam.

9. Fitur Nilai Tambah (Bonus)

- Upload foto bukti kerusakan saat membuat laporan.
- Timeline visual riwayat status laporan.
- Dashboard statistik dengan Chart.js / Recharts.
- Eskalasi otomatis berbasis background job (goroutine + ticker) — nilai tambah teknis utama yang membedakan sistem ini dari CRUD biasa.
- Badge notifikasi visual untuk laporan berprioritas tinggi.
- Rating/feedback dari pelapor setelah laporan berstatus selesai.
- Responsive layout dan dark mode.

10. Struktur Folder Project

10.1 Backend (Golang Fiber)

```
backend/
+-- config/
|   +-- database.go
|   +-- middleware/
+-- docs/                (swagger)
+-- handler/             (controller, request/response)
+-- model/               (struct GORM: users, kategori, laporan, riwayat)
+-- repository/          (query ke database)
+-- router/
```

```
+-- pkg/
|   +-- scheduler/      (cron job eskalasi otomatis)
+-- .env
+-- go.mod
+-- main.go
```

10.2 Frontend (Next.js App Router)

```
frontend/
+-- app/
|   +-- login/page.jsx
|   +-- register/page.jsx
|   +-- dashboard/page.jsx
|   +-- laporan/
|       |   +-- baru/page.jsx      (form buat laporan)
|       |   +-- [id]/page.jsx     (detail & timeline laporan)
|   +-- admin/
|       |   +-- kategori/page.jsx
|       |   +-- statistik/page.jsx
|   +-- layout.jsx
+-- components/
|   +-- atoms/
|   +-- molecules/
|   +-- organisms/      (DataTable, Timeline, StatCard, dll)
|   +-- layout/
+-- services/           (api.js, authService.js)
+-- lib/                (helper, validasi Zod)
+-- .env.local          (NEXT_PUBLIC_API_URL=...)
+-- package.json
```

11. Kesimpulan

SiLapor dirancang bukan sekadar sebagai aplikasi CRUD pelaporan, melainkan sebagai sistem yang secara aktif menjaga akuntabilitas penanganan fasilitas kampus melalui dua mekanisme kunci: distribusi otomatis laporan ke penanggung jawab yang tepat, dan eskalasi otomatis berbasis SLA yang mencegah laporan terbengkalai. Dengan kompleksitas teknis yang proporsional untuk dikerjakan oleh 2 orang dalam satu semester, sistem ini tetap memenuhi seluruh ketentuan teknis tugas besar (Golang Fiber, GORM, PostgreSQL Supabase, JWT, Swagger, role authorization) sekaligus menghadirkan nilai tambah nyata yang relevan dengan permasalahan sehari-hari di lingkungan kampus.